МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ДОНСКОЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(ДГТУ)

Факультет «Информатика и вычислительная техника»
Кафедра «Кибербезопасность информационных систем»

Лабораторная работа №5

по дисциплине: «Методы тестирования и отладки программного обеспечения»

На тему «Отладка программы путём трассировки с использованием logging»

Выполнил обучающийся гр. ВКБ43
Ковалев Данил Петрович

Проверил:
Ковальчик Р.В.

Ростов-на-Дону
2026

1. Тема, цель и постановка задачи

Тема работы: отладка Python-программы путём трассировки с использованием стандартного модуля logging.

Цель: воспроизвести вычислительную ошибку, локализовать её по промежуточным значениям в журнале, исправить причину и подтвердить результат повторным запуском и регрессионными тестами.

По заданию допускается любой небольшой проект с ошибкой. Для работы подготовлен учебный CLI расчёта заказа; дефект потери дробной части намеренно внесён в исходную версию legacy.py. Исправленный расчёт находится в order.py. Внешние сервисы и персональные данные не используются.

Исходные данные и ожидаемый результат

Позиция	Цена, ₽	Количество	Стоимость, ₽
Keyboard	1200,00	2	2400,00
Mouse	1600,00	1	1600,00
Сумма товаров	—	3 шт.	4000,00

Таблица 1 — заказ LAB5-001 из data/order.json

Скидка — 10%. Контрольный расчёт вручную: 4000 × (1 − 10 / 100) = 3600,00 ₽. Сумма скидки — 400,00 ₽. Значение expected_total хранится в JSON отдельно и используется только для проверки, а не при вычислении результата.

Среда выполнения: macOS, Python 3.14.3; зависимости зафиксированы в uv.lock. Расчёты выполняются с Decimal; окончательная сумма округляется до копеек по правилу ROUND_HALF_UP.

2. Воспроизведение ошибки

Сначала исходная версия запущена с уровнем INFO и ключом --check. Проверка сравнивает вычисленную сумму с независимым контрольным значением из входного файла.

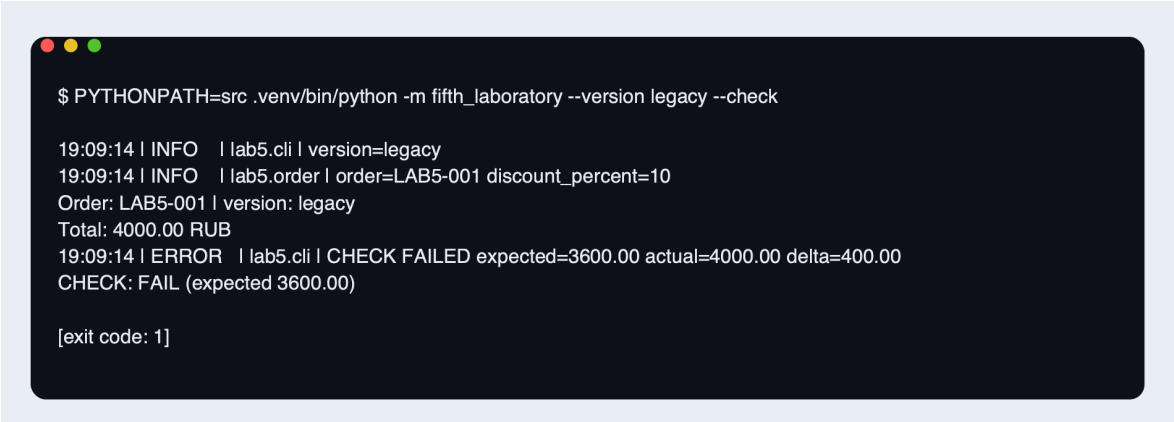


Рисунок 1 — исходная версия возвращает 4000,00 ₽ вместо 3600,00 ₽

Получен воспроизводимый дефект: программа завершается без исключения в расчёте, но результат завышен на 400,00 ₽. Сообщение ERROR относится к провалу контрольного сравнения; код завершения 1 задаётся ключом --check.

Гипотеза	Как проверить по журналу
1. Теряется дробная часть коэффициента	Вывести процент и коэффициент: ожидается 10 → 0.1.
2. Неверно складываются стоимости позиций	Сопоставить line_total и subtotal с ручным расчётом.
3. Скидка не вычитается из суммы	Сопоставить discount_amount и total.

Таблица 2 — гипотезы перед анализом трассировки

Уровень INFO фиксирует итог и исходный процент, но не показывает место искажения. Для различения гипотез включён DEBUG.

3. Настройка журнала и точек трассировки

Конфигурация logging задаётся один раз в CLI. StreamHandler направляет записи в stderr; FileHandler сохраняет тот же поток в UTF-8. Уровень выбирается аргументом --log-level. Формат содержит время, уровень, имя логгера и сообщение [1].

```python
import logging
from pathlib import Path

def configure_logging(level: str, log_file: Path | None) -> None:
    handlers: list[logging.Handler] = [logging.StreamHandler()]
    if log_file is not None:
        log_file.parent.mkdir(parents=True, exist_ok=True)
        handlers.append(logging.FileHandler(log_file, mode="w", encoding="utf-8"))
    logging.basicConfig(
        level=level,
        format="%(asctime)s | %(levelname)-7s | %(name)s | %(message)s",
        datefmt="%H:%M:%S",
        handlers=handlers,
        force=True,
    )
```

Рисунок 2 — фактическая конфигурация стандартного logging в __main__.py

Уровень	Применение в работе
DEBUG	Стоимость каждой позиции, subtotal, rate, discount_amount, total.
INFO	Выбранная версия, номер заказа, скидка и успешная проверка.
ERROR	Несовпадение с эталоном; ошибка входных данных с traceback.

Таблица 3 — назначение используемых уровней

Точки трассировки выбраны на границах вычислений: чтение скидки → сумма позиций → коэффициент → размер скидки → итог. Значения передаются через параметры logger.debug, поэтому код вычислений не подменяется диагностикой.

4. Анализ трассировки исходной версии

Тот же заказ повторно обработан с --log-level DEBUG. Полный журнал дополнительно записан в report_assets/logs/legacy.log; данные и контрольная сумма не изменялись.



Рисунок 3 — трассировка локализует ошибку при вычислении rate

Наблюдение в журнале	Вывод
line_total=2400.00 и 1600.00	Цена и количество обработаны верно.
subtotal=4000.00	Гипотеза 2 не подтверждена.
discount_percent=10 rate=0	Первое неверное значение: коэффициент скидки.
discount_amount=0.00 total=4000.00	Вычитание верно для ошибочно нулевой скидки; гипотеза 3 не подтверждена.

Таблица 4 — связь наблюдений с выводами

Подтвердилась гипотеза 1: данные и промежуточная сумма корректны, а ошибка появляется при переводе процентов в долю единицы. Дальнейший расчёт лишь распространяет уже искажённое значение.

5. Причина и исправление дефекта

В исходном выражении деление 10 / 100 даёт Decimal('0.1'), после чего int(...) отбрасывает дробную часть и возвращает 0. Обратное преобразование в Decimal уже не восстанавливает потерянную точность. Для любой скидки строго между 0% и 100% коэффициент становится нулевым.

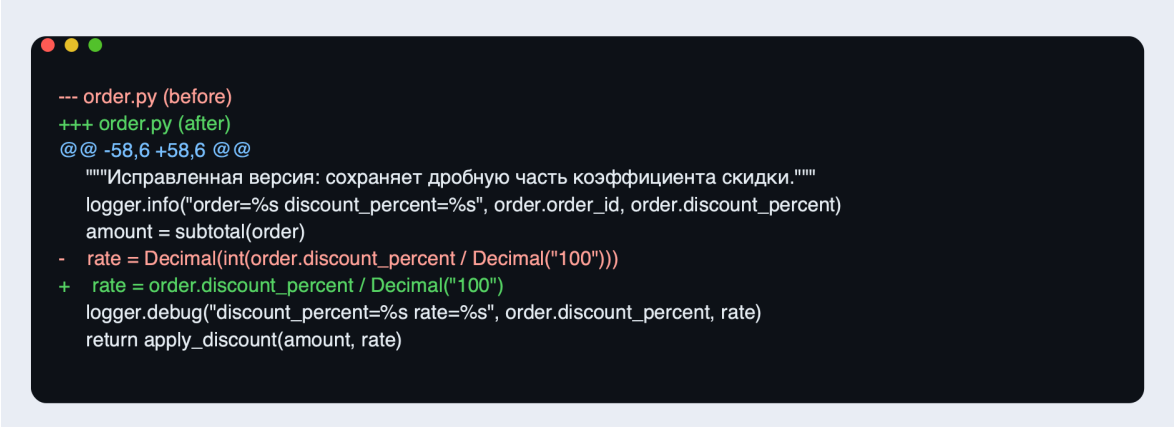


Рисунок 4 — исправление одной строки: удалено преобразование через int

Исправление сохраняет результат деления как Decimal: rate = discount_percent / Decimal('100'). Для исходного заказа rate = 0.1, discount_amount = 400.00 и total = 3600.00. Decimal выбран для точных десятичных входных значений [2]; переход на двоичный float для исправления не требуется.

Почему проверки только крайних значений недостаточно

При скидке 0% коэффициент равен 0 в обеих версиях; при 100% — 1 в обеих версиях. Поэтому эти два случая не обнаруживают дефект. Нужны как минимум промежуточная скидка 10% и дробная скидка 0,5%.

Область изменения ограничена преобразованием процентов в коэффициент. Формула суммы товаров и вычитание скидки оставлены общими для обеих версий. Исходный дефект сохранён в legacy.py, чтобы его можно было повторно продемонстрировать.

6. Регрессионная проверка до и после

Тест test_discount_regression проверяет один и тот же заказ и ожидает Decimal('3600.00'). Перед исправлением он завершился ошибкой сравнения 4000.00 и 3600.00. После изменения строки тот же тест проходит.



Рисунок 5 — регрессионный тест обнаруживает ошибку до исправления

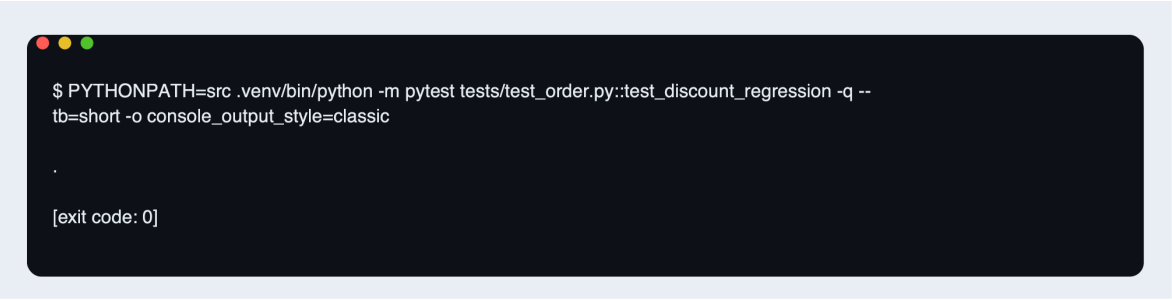


Рисунок 6 — тот же тест после исправления проходит

Для повторной сборки иллюстрации «до» создаётся изолированный снимок исходников в /tmp с прежней строкой вычисления rate. Тесты копируются без изменений. Рабочая исправленная версия при этом не меняется.

7. Повторный запуск исходного сценария

Исправленная версия обработала исходный data/order.json с теми же 10% скидки и контрольным значением 3600.00. Журнал сохранён в report_assets/logs/fixed.log.



Рисунок 7 — корректная трассировка и успешное контрольное сравнение

Показатель	До исправления	После исправления
subtotal, ₽	4000,00	4000,00
rate	0	0.1
discount_amount, ₽	0,00	400,00
total, ₽	4000,00	3600,00
Проверка / код завершения	FAIL / 1	PASS / 0

Таблица 5 — сравнение одного сценария на двух версиях

Логи подтвердили не только правильный итог, но и восстановление промежуточных значений. Поэтому результат не является случайным совпадением с контрольной суммой.

8. Автоматическое тестирование

Полный набор содержит 19 тестовых случаев: 16 для расчёта и трассировки, 3 для CLI. Все проверки завершились успешно; пропущенных и ожидаемо падающих тестов нет. Проверка legacy отдельно подтверждает сохранение исходного дефекта.

```
$ PYTHONPATH=src .venv/bin/python -m pytest -vv --tb=short -o console_output_style=classic

========================================= test session starts ==========================================
platform darwin -- Python 3.14.3, pytest-9.1.1, pluggy-1.7.0 --
/Users/arti/PycharmProjects/DSTU_VKB/testing_and_debugging_methods/2025/5/.venv/bin/python
cachedir: .pytest_cache
rootdir: /Users/arti/PycharmProjects/DSTU_VKB/testing_and_debugging_methods/2025/5
configfile: pyproject.toml
testpaths: tests
collecting ... collected 19 items

tests/test_cli.py::test_cli_check_and_log_file[legacy-1-CHECK: FAIL] PASSED
tests/test_cli.py::test_cli_check_and_log_file[fixed-0-CHECK: PASS] PASSED
tests/test_cli.py::test_cli_invalid_input PASSED
tests/test_order.py::test_discount_regression PASSED
tests/test_order.py::test_legacy_reproduces_defect PASSED
tests/test_order.py::test_discount_boundaries[0-4000.00] PASSED
tests/test_order.py::test_discount_boundaries[0.5-3980.00] PASSED
tests/test_order.py::test_discount_boundaries[10-3600.00] PASSED
tests/test_order.py::test_discount_boundaries[25-3000.00] PASSED
tests/test_order.py::test_discount_boundaries[100-0.00] PASSED
tests/test_order.py::test_rounding_to_kopeck PASSED
tests/test_order.py::test_invalid_discount[-1] PASSED
tests/test_order.py::test_invalid_discount[101] PASSED
tests/test_order.py::test_invalid_discount[NaN] PASSED
tests/test_order.py::test_invalid_quantity[0] PASSED
tests/test_order.py::test_invalid_quantity[-1] PASSED
tests/test_order.py::test_invalid_quantity[1.5] PASSED
tests/test_order.py::test_invalid_quantity[True] PASSED
tests/test_order.py::test_trace_contains_intermediate_values PASSED

========================================== 19 passed in 0.14s ==========================================

[exit code: 0]
```

Рисунок 8 — полный запуск pytest: 19 passed

Проверены скидки 0%, 0,5%, 10%, 25%, 100%; округление 0.045 → 0.05; отклонение скидок −1%, 101% и NaN, а также недопустимых количеств. Дополнительно проверены промежуточные DEBUG-записи, запись журнала в файл, коды завершения CLI и traceback при отсутствии входного файла.

9. Выводы и воспроизведение

Выполнена отладка учебного Python-проекта с помощью стандартного logging. Ошибка проявлялась неверной суммой заказа без исключения. Трассировка позволила установить первое неверное значение — rate=0 после преобразования через int. Удаление этого преобразования восстановило скидку 400,00 ₽ и итог 3600,00 ₽.

Повторный запуск исходного сценария и все 19 тестов подтвердили исправление. Для предотвращения повторения дефекта необходимо проверять промежуточные проценты и дробные скидки. Подробные сообщения оставлены на DEBUG; по умолчанию используется INFO.

Команды из каталога лабораторной №5

```
uv sync --all-groups
export PYTHONPATH=src
# Воспроизведение дефекта: код завершения 1
.venv/bin/python -m fifth_laboratory \
  --version legacy --log-level DEBUG --check
# Исправленная версия: код завершения 0
.venv/bin/python -m fifth_laboratory \
  --version fixed --log-level DEBUG --check
.venv/bin/python -m pytest
# Обновление иллюстраций и DOCX
.venv/bin/python scripts/capture_evidence.py
.venv/bin/python scripts/build_report.py
```

Источники

1. Python Software Foundation. logging — Logging facility for Python. https://docs.python.org/3/library/logging.html (дата обращения: 20.10.2026).

2. Python Software Foundation. decimal — Decimal fixed-point and floating-point arithmetic. https://docs.python.org/3/library/decimal.html (дата обращения: 20.10.2026).

3. Условие лабораторной работы №5 (README.md); исходники и журналы запусков настоящего проекта.